

Structure your Node.js service the way senior engineers do.

Five lenses, six axioms, one stack — a one-page-per-lens playbook of the architectural decisions that separate a Node.js developer from a Node.js architect.

By **Petar Ivanov** · petarivanov.tech

HOW JUNIOR & SENIOR ENGINEERS READ A CODEBASE

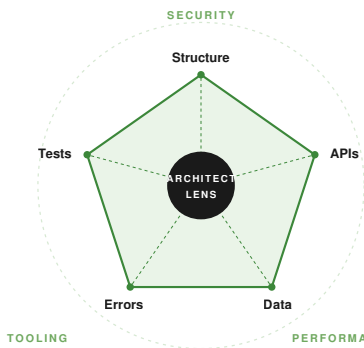
JUNIOR READS

Opens `src/`. Reads file names. Searches for `server.ts`. Follows `app.get(...)` calls top to bottom.

SENIOR READS

Opens `src/`. Reads module boundaries. Asks "who owns this data, who validates it, where do failures surface?"

THE FIVE LENSES



A senior engineer reads any Node.js codebase through five inner lenses. **Security**, **performance**, and **tooling** sit on the outer ring — they exist to support the choices made above. Optimize the inner five first; the outer ring follows.

Each page that follows takes one lens deep. Read in order, or jump to the lens you're worst at.

SIX ARCHITECT AXIOMS

- 01

Module by domain, not by tech. A feature is one folder — not files scattered across `routes/`, `services/`, `repos/`.
- 02

Validate at every trust boundary. Zod-parse every body, query, env var, and external response before it reaches your domain.
- 03

Server state has a repository. Domain code speaks in entities, not rows — swap Postgres for in-memory in tests.
- 04

Idempotency on every write. Clients send `Idempotency-Key`; the server replays the stored response on retries.
- 05

Don't block the event loop. Offload CPU work, stream large payloads, never sync-read or sync-hash.
- 06

Pick tools, not platforms. Each tool does one thing well and composes with the rest. No lock-in.

STACK BASELINE

RUNTIME	Node.js LTS (24 / 22)	Even-numbered releases with long-term support. Skip Bun for production services.
LANGUAGE	TypeScript	Strict mode, no any, no implicit returns. Catches a third of bugs before runtime.
PKG MGR	pnpm	Content-addressed store, strict graph, fast installs. <code>pnpm install --frozen-lockfile</code> in CI.
FRAMEWORK	Express 5	Default. Async errors, lean, the largest ecosystem. Fastify for throughput, NestJS for structured DI.
SCHEMAS	Zod	Validate at every trust boundary — request bodies, query params, env vars, and external API responses.

```
// schemas/User.ts – one contract that becomes the truth everywhere downstream
export const User = z.object({ id: z.string().uuid(), email: z.string().email() })
```

Each tool does one thing well and composes cleanly with the others. None tries to be a platform. None lock you in.

Folder structure & module boundaries

A folder structure is a contract. It says *"these files belong together; these don't."* Get it right and the codebase guides every future change. Get it wrong and engineers fight the tree before they touch a single route.

ANNOTATED FEATURE-BASED TREE

```
src/
├── modules/                domain modules – one folder per bounded context
│   ├── checkout/
│   │   ├── checkout.routes.ts    HTTP surface – Express router
│   │   ├── checkout.service.ts   business logic, pure of HTTP
│   │   ├── checkout.repo.ts      Kysely / Objection – data edge
│   │   ├── checkout.schema.ts    Zod schemas at the boundary
│   │   ├── checkout.test.ts      integration test, co-located
│   │   └── index.ts              module's public surface
│   └── profile/
├── common/                 cross-cutting code – the only shared target
│   ├── db.ts                pool + Kysely instance
│   ├── logger.ts            pino with redaction
│   ├── errors.ts            DomainError / SystemError taxonomy
│   └── otel.ts              OpenTelemetry SDK, started once
├── app.ts                  Express bootstrap, middleware order
└── server.ts               graceful start & shutdown
```

Five rules the tree enforces

- 01 Group by domain, not by tech.** Modules own their routes, services, repos, and tests.
- 02 Cross-module talk goes through services.** No `checkout` → `profile/repo` imports — call the exposed service.
- 03 If two modules need it, lift to `common/`.** Logger, db pool, error taxonomy, OTel.
- 04 Co-locate by use.** Helpers used by one module stay inside that module.
- 05 Wrap third-party libs.** Adapt them in `common/` so swapping is one file's problem.

LAYERED VS. FEATURE-BASED VS. HEXAGONAL

LAYERED

Group by tech

Top-level `controllers/`, `services/`, `models/`. Everyone knows where things go.

Use when: small service, single team, < 5 endpoints.

Breaks at: a feature bleeds across four folders — one change, four trees.

FEATURE-BASED

Group by domain

One folder per bounded context owns its routes, services, repos, schemas, and tests.

Use when: mid-large monolith, multiple teams, change locality matters.

Breaks at: early-stage prototypes — ceremony before insight.

HEXAGONAL

Ports & adapters

Domain core depends on *interfaces*; adapters live at the edge (HTTP, DB, queue, fakes).

Use when: long-lived services that swap infra (Postgres → DynamoDB, REST → gRPC).

Watch for: over-abstracted seams that no one ever swaps.

TRADEOFF — THE COMMON/ TRAP

"Shared" code used by exactly one module is just deferred refactoring. Run a quarterly pruning pass: anything imported by one feature moves back into that feature. Keep `common/` for what's *actually* cross-cutting — logger, db, errors, config.

THE TAKEAWAY

✖ AVOID

A top-level `controllers/`, `services/`, `repositories/` tree once you have more than a handful of routes — every change becomes a multi-folder hunt.

✔ PREFER

A feature-based `modules/` tree with a single `common/` — modules import from themselves or from `common/`, never from each other's internals.

API design: REST & GraphQL contracts

An API is a contract you can't take back. Versioning a route is cheap before clients exist and ruinous after. Senior engineers design APIs as products — predictable shapes, safe retries, validated inputs, and one direction of data flow from edge to domain.

REQUEST FLOW — ONE DIRECTION ONLY

```
router → middleware (parse · auth · validate) → service → repository → db
```

Each layer hides the one below. Routes never call repositories. Services never see the request object. Schemas live at the edge, not in the domain.

MIDDLEWARE ORDER — DELIBERATE, NOT ACCIDENTAL

```
helmet · cors · body-parser · request-id · pino-http · rate-limit · auth · validate(zod) · routes · 404 · error-handler
```

Security headers first, error handler last. The 404 catcher sits between the last route and the error handler so unmatched paths land in the same funnel.

REST PATTERNS

- **Version the path.** `/api/v1/...` from day one. Breaking changes ship as `/v2` with overlap.
- **One response shape.** `{ data, error, meta }`, every endpoint, every time.
- **Idempotency-Key on every write.** Server stores the response keyed by it; retries replay.
- **Cursor pagination > offset.** Stable under inserts; cheap on indexed columns.
- **Health / ready split.** `/health` for liveness, `/ready` for "I can serve traffic".

GRAPHQL PATTERNS

- **DataLoader per request.** Batch sibling queries — kills the N+1 in resolvers.
- **Persisted queries in production.** Ship a sha256 registry; reject anything not in it.
- **Typed mutation payloads.** `{ entity, errors, warnings }` with a union error type — no thrown exceptions across the wire.
- **Global IDs (Relay).** `base64(Type:DbId)` so any ID round-trips through `node(id:)`.

VALIDATE AT THE EDGE — ZOD + EXPRESS

```
// modules/checkout/checkout.schema.ts
export const CreateOrder = z.object({
  userId: z.string().uuid(),
  items: z.array(z.object({ sku: z.string(), qty: z.number().int().positive() })).min(1),
})

// modules/checkout/checkout.routes.ts
router.post('/orders', validate({ body: CreateOrder }), async (req, res) => {
  const order = await checkoutService.createOrder(req.body, { idempotencyKey: req.header('idempotency-key') })
  res.json({ data: order, error: null, meta: { requestId: req.id } })
})
```

Once `req.body` is parsed, the service receives a typed, guaranteed shape — no defensive checks downstream.

TRADEOFF — EXPRESS VS. FASTIFY VS. NESTJS

Default to **Express 5** — async errors are native and the ecosystem is the largest. Reach for **Fastify** when throughput is the constraint (its schema-based validation is faster than middleware). Reach for **NestJS** when team size demands enforced structure — DI containers, decorators, module boundaries out of the box.

THE TAKEAWAY

✗ AVOID

Putting business logic in route handlers and inventing a new response shape per endpoint — every client integration becomes a special case, every retry becomes a duplicate write.

✓ PREFER

Routes that validate, delegate, and serialize. A single response envelope. Idempotency keys on writes. The domain never sees an HTTP object.

Data: persistence, caching & transactions

The data layer is where speed, correctness, and safety either compose — or collide. The senior move: **name the data edge with a repository**, validate at the boundary, cache deliberately, and publish events with an outbox so writes and notifications can't drift apart.

WHERE DOES THIS DATA LIVE?

Domain entities (orders, users, accounts)?	→ Postgres via repository	<i>alt: MySQL · CockroachDB</i>
Hot, frequently-read, short-TTL data?	→ Redis (TTL 5–60s)	<i>alt: Memcached</i>
Sessions, rate-limit counters, locks?	→ Redis – never in-memory	<i>keeps app stateless</i>
Reference data that rarely changes?	→ In-process LRU cache	<i>e.g. lru-cache</i>
Files, blobs, exports?	→ S3 / object store	<i>never in your db</i>
Cross-service event distribution?	→ Transactional outbox + worker	<i>not dual-writes</i>

QUERY BUILDER VS. ORM — THE RIGHT TOOL PER QUERY

QUERY BUILDER · KYSELY

Raw SQL with autocomplete and compile-time safety. Use when the query is the thing — joins, CTEs, window functions, reporting queries. You read what runs.

ORM · OBJECTION / PRISMA

Use when the entity has behavior — relations to traverse, lifecycle hooks, schema validation. `withGraphFetched` beats five hand-written joins.

THE REPOSITORY AT THE DATA EDGE

```
// modules/checkout/checkout.repo.ts
export const checkoutRepo = (db: Kysely<DB>) => ({
  findById: (id: string) =>
    db.selectFrom('orders').selectAll().where('id', '=', id).executeTakeFirst()
    .then(row => row ? Order.parse(row) : null), // Zod → typed entity at the edge

  create: (cmd: CreateOrderCmd, trx?: Trx) => (trx ?? db)
    .insertInto('orders').values(toRow(cmd)).returningAll().executeTakeFirstOrThrow(),
})
```

Domain code speaks in Order entities, not orders rows. Tests swap the Kysely instance for an in-memory adapter — business logic doesn't notice.

TRANSACTIONAL OUTBOX — ATOMIC WRITES & EVENTS

```
// inside one DB transaction:
await db.transaction().execute(async trx => {
  const order = await checkoutRepo(trx).create(cmd)
  await outboxRepo(trx).enqueue({
    type: 'order.placed',
    payload: order,
  })
})
// a separate worker polls the outbox and
// publishes — at-least-once, in order.
```

Three moves to know

- **Migrations ship first.** Expand → migrate → deploy code → contract. Schema PRs land ahead of code PRs.
- **Pool everything.** `pg.Pool` sized to `cores × 2–4`. Never new `Client()` per request.
- **Index what you query, not everything.** Add an index from a slow log; never speculatively.

TRADEOFF — CACHING FRESHNESS VS. COMPLEXITY

Every cache is a correctness bet: how stale can this data be? Start with TTL caching (60s) for reads, invalidate on writes for hot keys. Reach for write-through caches only when staleness is intolerable *and* you've measured the read pressure.

THE TAKEAWAY

✖ AVOID

Calling the database from a route handler, then "publishing" an event with a separate write — you've built a dual-write that will silently desync the moment one side fails.

✔ PREFER

A repository that returns typed entities and an outbox that ships the event in the same transaction. One commit, one truth, one place to debug when something diverges.

Errors, logging & observability

In a Node.js service, the question isn't *if* something fails — it's whether you'll know what failed, where, and to which request. Senior engineers design for the 3 AM page: typed errors that classify themselves, structured logs you can grep, a correlation ID on every line, and one OpenTelemetry SDK that gives you traces, metrics, and logs in the same pane.

THE SPLIT THAT MATTERS

OPERATIONAL ERRORS

Expected outcomes of a healthy program — a 404, a validation failure, a timeout from an upstream. **Handle them.** Map to a response, log, move on.

PROGRAMMER ERRORS

Bugs — null where there shouldn't be one, an invariant broken. **Crash.** Let the process manager restart you in a clean state.

TYPED ERROR TAXONOMY

```
export class AppError extends Error {
  constructor(public code: string, public status: number, msg: string, public ctx?: object) { super(msg) }
}
export class DomainError extends AppError {} // 4xx – user/business problem
export class SystemError extends AppError {} // 5xx – our problem, page someone

export class NotFound extends DomainError { constructor(what: string){ super('NOT_FOUND', 404, what) } }
export class UpstreamFail extends SystemError { constructor(svc: string){ super('UPSTREAM', 502, svc) } }
```

The error handler reads `err.status` and `err.code`. Clients see safe messages; logs see the full `ctx`.

STRUCTURED LOGS + CORRELATION IDS

```
// common/logger.ts – pino, JSON to stdout, redaction baked in
export const logger = pino({ redact: ['req.headers.authorization', '*.password', '*.token'] })

// middleware – thread one request-id end to end
app.use(pinoHttp({ logger, genReqId: req => req.header('x-request-id') ?? randomUUID() }))
app.use((req, res, next) => { res.setHeader('x-request-id', req.id); next() })
```

One `requestId` per log line, returned in every response, propagated on every outbound HTTP call. You can rebuild any incident from grep.

THREE PILLARS — WIRE OPENTELEMETRY ONCE

- **Traces.** Span every external call — DB query, HTTP fetch, queue publish. The flamegraph tells you which hop is slow.
- **Metrics.** `http.request.duration`, error rate by code, event-loop lag. RED + USE in one dashboard.
- **Logs.** Same correlation ID. OTLP exporter, your backend's choice (Tempo, Honeycomb, Datadog).

GRACEFUL SHUTDOWN — THE ORDER

SIGTERM → stop accepting new connections → drain in-flight requests (with timeout) → close DB pool → flush logger → `process.exit(0)`. Skip any step and you'll lose responses mid-deploy.

TRADEOFF — WHAT NOT TO LOG

Never log secrets, full tokens, raw passwords, full PII, or large request bodies. Configure Pino's `redact` path before you ship — fixing a leak after it's in your log aggregator means rotating every credential that was ever logged.

THE TAKEAWAY

✗ AVOID

`console.log` scattered through handlers, a single `Error` class everywhere, and a 5xx response with no request ID — every incident becomes archaeology.

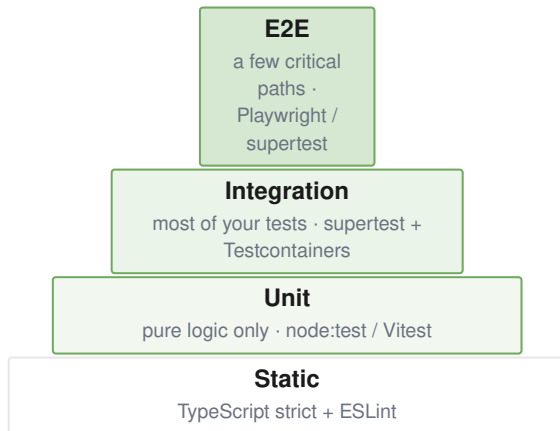
✓ PREFER

A typed error taxonomy that maps to status codes, structured JSON logs with a threaded request ID, and one OpenTelemetry SDK wired into all three pillars.

Testing strategy

Tests are an architecture decision. A thousand unit tests that mock the database tell you your mocks work; a hundred integration tests that hit a real Postgres tell you your *system* works. Pick the right shape of pyramid and a small, fast suite catches almost everything that ships to production.

THE SHAPE OF A HEALTHY NODE TEST SUITE



What to test where

- **Integration is the bulk.** Boot the app, hit it with `supertest`, point it at a real Postgres via Testcontainers. Asserts what the API actually returns.
- **Unit for pure logic only.** Calculators, validators, pricing rules — the deterministic stuff with no I/O.
- **E2E for revenue paths.** Signup, checkout, payment — the workflows where a regression costs money.
- **Contracts for service-to-service.** Pact consumer-driven contracts catch breakage in the producer's CI.
- **Static is free.** Strict TypeScript + ESLint catches a third of bugs before a test ever runs.

RECOMMENDED STACK

RUNNER	node:test · Vitest	Native runner ships with Node 18+. Vitest if you want watch mode & coverage UX out of the box.
HTTP	supertest	Boot the Express/Fastify app in-process, fire real HTTP — no mocking the framework.
DB	Testcontainers	Real Postgres in Docker for the test run. Migrations run on boot. No <code>jest.mock('pg')</code> .
CONTRACTS	Pact	Consumer publishes the expected contract; producer's CI replays it. Breakage caught before deploy.
E2E	Playwright	Real browsers for SSR flows; reuse supertest for headless API journeys.

INTEGRATION TEST — SUPERTEST + TESTCONTAINERS

```

it('POST /orders creates an order and returns 201', async () => {
  const res = await request(app)
    .post('/api/v1/orders')
    .set('idempotency-key', randomUUID())
    .send({ userId: testUser.id, items: [{ sku: 'SKU-1', qty: 2 }] })

  expect(res.status).toBe(201)
  expect(res.body.data).toMatchObject({ status: 'placed' })
  expect(await db.selectFrom('outbox').execute()).toHaveLength(1) // event was enqueued
})
  
```

Test names — describe behavior

- ✓ "returns 404 when the order does not exist"
- ✓ "replays the stored response on duplicate key"
- ✗ "calls orderRepo.create once"

Anti-patterns to drop

- **Mocking your own database.** You're testing the mock, not the query.
- **Coverage chasing.** 100% of trivial branches buys nothing.
- **Snapshots as default.** They flag noise, not bugs.

THE TAKEAWAY

✗ AVOID

A wall of unit tests that mock `pg`, `axios`, and every collaborator — green numbers, zero confidence on release day, and a suite that breaks on every refactor.

✓ PREFER

Many integration tests against a real Postgres via Testcontainers, a few E2E flows on revenue paths, unit tests reserved for pure business logic.

Security, performance, tooling & the 10 rules

The outer ring exists to support the five lenses — secure the inputs, keep the event loop free, automate the chores. Then the closing manifesto: ten heuristics every senior Node engineer can recite from memory.

SECURITY — THE NON-NEGOTIABLES

- **Validate every input.** Zod at the boundary — body, query, params, headers, env.
- **Parameterized queries only.** No string concatenation reaches the driver.
- **Hash passwords with argon2id.** Never bcrypt-with-default-rounds, never SHA-anything.
- **Tokens in httpOnly cookies.** Short JWT expiry; rotate with refresh tokens.
- **Helmet + rate limit.** CSP, HSTS, X-Frame-Options; throttle by IP & by user.
- **Constant-time compare.** `crypto.timingSafeEqual` for API keys & HMACs.
- **Run as non-root.** `USER node` in your Dockerfile.
- **npm audit in CI.** Renovate for grouped patch PRs.

PERFORMANCE — IN THIS ORDER

- 01 **Don't block the event loop.** No `readFileSync`, no sync hashing, no CPU loops in handlers.
- 02 **Stream large payloads.** Pipes from S3 / DB straight to the response.
- 03 **Profile before you optimize.** `--inspect`, `clinic.js`, `0x`. Never optimize on a hunch.
- 04 **All cores, no plus one.** Cluster mode or container replicas — one process per core.
- 05 **Worker threads for CPU.** Offload hashing, image work, parsing.
- 06 **Bound every network call.** `AbortSignal.timeout`; `outbound < DB < inbound`.
- 07 **Retry + circuit breaker.** Backoff with jitter; `opossum` when the dep stays down.

QUALITY AUTOMATION — EVERY COMMIT, NO EXCEPTIONS

FORMAT	Biome · Prettier	Biome is one binary for lint + format, ~10× faster. Prettier stays where the plugin ecosystem matters.
LINT	Biome · ESLint	Biome out of the box; ESLint if you need <code>plugin:security</code> or framework-specific rules.
HOOKS	Husky + lint-staged	Format, lint, type-check the staged files on every commit. No "I'll fix it in CI."
DEPS	Renovate · Dependabot	Grouped patch PRs, auto-merge for safe bumps, security advisories surface as PRs.
BUILD	tsx (dev) · tsup (build)	Zero-config TypeScript runner for dev, esbuild-backed bundler for production.
CI GATE	GitHub Actions	<code>format · lint · type · test · audit</code> — every commit. Failing CI blocks merge.

THE 10 RULES OF SENIOR NODE

- 01 **Module by domain, not by tech.** A feature is one folder.
- 02 **Repository at the data edge.** Domain code speaks in entities, not rows.
- 03 **Validate at every trust boundary** with Zod. Bodies, queries, env, external responses.
- 04 **Idempotency on every write.** Clients send the key; you store the result.
- 05 **Don't block the event loop.** Offload, stream, never sync.
- 06 **Structured logs with correlation IDs.** JSON to stdout, request ID end to end.
- 07 **Outbox before two-phase commit.** State change and event in one transaction.
- 08 **Trace every cross-service hop.** One OTel SDK, OTLP exporter, span every external call.
- 09 **Retry with backoff + circuit breaker.** Never retry 4xx. Bounded attempts, bounded delay.
- 10 **Automate every quality gate.** Format, lint, type, test, audit. Husky + CI, no exceptions.

The best stack is the one you stop noticing — until it saves you.

Advance your career with production-grade patterns and real-world examples on my newsletter — [petarivanov.tech](#). The path from Node developer to Node architect is the path from "I can build this" to "I can decide what to build, how to scale it, and why one architecture beats another."

Created by [Petar Ivanov](#) · [linkedin.com/in/petarivanovv9](#) · [petarivanov.tech](#)